Formal Verification: Revocation Model

Zach Kelling, Lux Industries

December 2025

Abstract

We prove that revocations are monotone (adding revocations only removes trust paths), signature-verified (unsigned revocations are ignored), and composable (filtering to valid revocations preserves revocation status).

1 Introduction

The revocation model ensures that trust paths can be explicitly canceled. Revocations are cryptographically signed to prevent denial-of-service attacks on the trust graph. The monotonicity property ensures that revocations are permanent.

2 Definitions

Definition 1 (Revocation)**.** *A revocation record: revoker, revoked, timestamp, signature validity.*

Definition 2 (VouchRef)**.** *A vouch reference: voucher and vouchee.*

3 Theorems and Axioms

Theorem 3 (`unsigned_revocation_ignored`)**.** *Revocations with invalid signatures are ignored.*

Theorem 4 (`revocation_monotone`)**.** *Adding a revocation preserves existing revocations: if vouch was revoked before, it remains revoked.*

Proof. By `List.any_cons` and `Or.inr`. □

Theorem 5 (`empty_revokes_nothing`)**.** *An empty revocation list revokes nothing.*

Theorem 6 (`valid_revocation_works`)**.** *A correctly signed revocation for the exact vouch succeeds.*

Theorem 7 (`valid_filter_preserves`)**.** *Filtering to only valid (signed) revocations preserves revocation status.*

4 Lean Source

```
/-
  Revocation Model

  Revocations cancel vouches. A revocation must be cryptographically
  signed by the revoker - otherwise anyone could DoS the trust graph.

  Maps to:
  - hanzo/iam: token revocation lists
  - lux/node: validator key rotation and revocation

  Properties:
  - Revocations are signature-verified
  - Revocation is monotone (adding revocations only removes trust paths)
  - Revoked vouches have no authority
-/

import Mathlib.Data.Nat.Defs
import Mathlib.Data.List.Basic
import Mathlib.Tactic

namespace Trust.Revocation

/-- A revocation record -/
structure Revocation where
  revoker : Nat      -- who is revoking
  revoked : Nat      -- whose vouch is being revoked
  timestamp : Nat    -- when
  sigValid : Bool    -- has the signature been verified?

/-- A vouch reference (simplified) -/
structure VouchRef where
  voucher : Nat
  vouchee : Nat

/-- Check if a vouch is revoked -/
def isRevoked (revocations : List Revocation) (v : VouchRef) : Bool :=
  revocations.any fun r =>
    r.sigValid && r.revoker == v.voucher && r.revoked == v.vouchee

/-- Unsigned revocations are ignored -/
theorem unsigned_revocation_ignored (r : Revocation) (v : VouchRef)
    (h : r.sigValid = false) :
    ¬(r.sigValid && r.revoker == v.voucher && r.revoked == v.vouchee) = true
       := by
  simp [h]

/-- Adding a revocation can only increase the revoked set (monotone) -/
theorem revocation_monotone (revocations : List Revocation) (r : Revocation) (
   v : VouchRef)
    (h : isRevoked revocations v = true) :
    isRevoked (r :: revocations) v = true := by
  simp [isRevoked, List.any_cons] at *
```

```
  exact Or.inr h

/-- Empty revocation list revokes nothing -/
theorem empty_revokes_nothing (v : VouchRef) :
    isRevoked [] v = false := rfl

/-- A valid revocation for the exact vouch works -/
theorem valid_revocation_works (revoker revoked ts : Nat) (v : VouchRef)
    (h_revoker : revoker == v.voucher = true)
    (h_revoked : revoked == v.vouchee = true) :
    isRevoked [revoker, revoked, ts, true] v = true := by
  simp [isRevoked, List.any_cons, List.any_nil, h_revoker, h_revoked]

/-- Filter: only valid (signed) revocations matter -/
def validRevocations (revocations : List Revocation) : List Revocation :=
  revocations.filter (·.sigValid)

/-- Filtering preserves revocation status -/
theorem valid_filter_preserves (revocations : List Revocation) (v : VouchRef)
    (h : isRevoked revocations v = true) :
    isRevoked (validRevocations revocations) v = true := by
  simp [isRevoked, validRevocations, List.any_eq_true, List.mem_filter] at *
  obtain r, hr_mem, hr_match := h
  simp [Bool.and_eq_true] at hr_match
  exact r, hr_mem, hr_match.1, by simp [hr_match]

end Trust.Revocation
```

5 References

Source code: https://github.com/luxfi/proofs/blob/main/lean/Trust/Revocation.lean
White papers: https://papers.lux.network
Related white papers:

- lux-id-iam.tex